

Vel Tech Group of Institutions

Name: GOPICHAND KAKIVAI

Email: vtu29748@veltech.edu.in

Roll no: vtu29748

Phone: null

Branch: VTU

Department: CSE

Batch: 2028

Degree: B.Tech - Computer Science and Engineering

Scan to verify results



2028_NeoColab_Design and Analysis of AlgorithmsS1L3

VTU_DAA_Week 9_CY

Attempt : 1

Total Mark : 50

Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Given an adjacency matrix $adj[][]$ of an undirected graph consisting of N vertices, find a Hamiltonian Path in the graph if one exists. If found, print the path as a sequence of vertex indices separated by $\rightarrow$. Otherwise, print No Hamiltonian Path found.

A Hamiltonian path visits each and every vertex of the graph exactly once. The adjacency matrix is symmetric with zeros on the diagonal (no self-loops).

Write a program using Backtracking to find the Hamiltonian Path.

Examples:

Input: $adj[][] = \{\{0, 1, 1, 1, 0\}, \{1, 0, 1, 0, 1\}, \{1, 1, 0, 1, 1\}, \{1, 0, 1, 0, 0\}\}$

Output: Yes

Explanation:

There exists a Hamiltonian Path for the given graph, as shown in the image below:

Input: $\text{adj}[][] = \{\{0, 1, 0, 0\}, \{1, 0, 1, 1\}, \{0, 1, 0, 0\}, \{0, 1, 0, 0\}\}$

Output: No

Input Format

The first line contains an integer N, representing the number of vertices in the graph.

The next N lines each contain N space-separated integers (0 or 1), representing the adjacency matrix of the graph.

Output Format

If a Hamiltonian path exists, print the vertices of the path separated by -> on a single line.

If no Hamiltonian path exists, print: No Hamiltonian Path found

Refer to the sample output for formatting specifications.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 5

0 1 1 1 0

1 0 1 0 1

1 1 0 1 1

1 0 1 0 0

0 1 1 0 0

Output: 0 -> 1 -> 4 -> 2 -> 3

Answer

```
#include <stdio.h>
```

```
int N;
int adj[10][10];
int path[10];
int visited[10];

int dfs(int pos) {
    if (pos == N) return 1;

    for (int v = 0; v < N; v++) {
        if (!visited[v] && adj[path[pos - 1]][v] == 1) {
            visited[v] = 1;
            path[pos] = v;

            if (dfs(pos + 1)) return 1;

            visited[v] = 0;
        }
    }
    return 0;
}
```

```
int main() {
    scanf("%d", &N);

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            scanf("%d", &adj[i][j]);
        }
    }

    for (int start = 0; start < N; start++) {
        for (int i = 0; i < N; i++) visited[i] = 0;

        path[0] = start;
        visited[start] = 1;

        if (dfs(1)) {
            for (int i = 0; i < N; i++) {
                printf("%d", path[i]);
                if (i != N - 1) printf("-> ");
            }
        }
    }
}
```

```

        return 0;
    }
}

printf("No Hamiltonian Path found");
return 0;
}

```

Status : Correct

Marks : 10/10

2. Problem Statement

You are given an undirected graph with N vertices. Implement a function `longest_hamiltonian_cycle` to find the longest Hamiltonian cycle in the graph. A Hamiltonian cycle is a cycle that visits every vertex exactly once, except for the starting and ending vertices, which are the same.

Input Format

The first line of input consists of an integer N representing the number of vertices in the graph.

The next N lines each contain N space-separated integers, representing the adjacency matrix of the graph. The value at `graph[i][j]` is 1 if there is an edge between vertex i and vertex j , and 0 otherwise.

Output Format

If a Hamiltonian cycle exists, print "Longest Hamiltonian Cycle:" followed by the vertices of the longest cycle found in order within square bracket separated by a comma.

If no Hamiltonian cycle exists, print "No Hamiltonian Cycle found".

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4
0 1 1 0

```
1 0 1 1
1 1 0 1
0 1 1 0
```

Output: Longest Hamiltonian Cycle: [0, 1, 3, 2]

Answer

```
#include <stdio.h>
```

```
int N;
int graph[10][10];
int path[10];
int visited[10];
int found = 0;
```

```
int isSafe(int v, int pos) {
    if (!graph[path[pos - 1]][v]) return 0;
    if (visited[v]) return 0;
    return 1;
}
```

```
int checkCycle() {
    return graph[path[N - 1]][path[0]];
}
```

```
int solve(int pos) {
    if (pos == N) {
        if (checkCycle()) {
            found = 1;
            return 1;
        }
        return 0;
    }
}
```

```
for (int v = 0; v < N; v++) {
    if (isSafe(v, pos)) {
        path[pos] = v;
        visited[v] = 1;

        if (solve(pos + 1)) return 1;

        visited[v] = 0;
    }
}
```

```

    }
    return 0;
}

int main() {
    scanf("%d", &N);

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            scanf("%d", &graph[i][j]);
        }
    }

    for (int start = 0; start < N; start++) {
        for (int i = 0; i < N; i++) visited[i] = 0;

        path[0] = start;
        visited[start] = 1;

        if (solve(1)) break;
    }

    if (!found) {
        printf("No Hamiltonian Cycle found");
    } else {
        printf("Longest Hamiltonian Cycle: [");
        for (int i = 0; i < N; i++) {
            printf("%d", path[i]);
            if (i != N - 1) printf(", ");
        }
        printf("]");
    }

    return 0;
}

```

Status : Correct

Marks : 10/10

3. Problem Statement

Nira is studying the N-Queens problem and needs to write a program to solve it. The program should find a valid arrangement of N queens on an $N \times N$ chessboard, ensuring that no two queens can attack each other.

Two queens attack each other if they are in the same row, the same column, or the same diagonal. Once a solution is found, display the board with queens placed correctly. A solution is guaranteed to exist for all valid values of N.

Write a program using Backtracking to solve the N-Queens problem.

Example

Input:

4 // Value of N

Output:

0 0 1 0

1 0 0 0

0 0 0 1

0 1 0 0

Explanation: The output represents a valid arrangement of queens on the 4×4 chessboard, where 1 indicates the position of a queen and 0 indicates an empty cell.

In the first row, a queen is placed in the third column. In the second row, a queen is placed in the first column. In the third row, a queen is placed in the fourth column. In the fourth row, a queen is placed in the second column. This arrangement of queens on the board ensures that no two queens threaten each other.

Input Format

The input consists of a single integer N, representing the size of the chessboard ($N \times N$).

Output Format

Print the N×N board as N rows of N space-separated integers. Each integer is followed by a space (including the last integer on each row). Print 1 where a queen is placed and 0 for an empty cell. Print any one valid solution.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 4

Output: 0 0 1 0

1 0 0 0

0 0 0 1

0 1 0 0

Answer

```
#include <stdio.h>
```

```
#include <stdbool.h>
```

```
int board[20][20];
```

```
bool isSafe(int row, int col, int N) {  
    int i, j;  
    for (i = 0; i < row; i++) {  
        if (board[i][col]) return false;  
    }  
    for (i = row, j = col; i >= 0 && j >= 0; i--, j--) {  
        if (board[i][j]) return false;  
    }  
    for (i = row, j = col; i >= 0 && j < N; i--, j++) {  
        if (board[i][j]) return false;  
    }  
    return true;  
}
```

```
bool solve(int row, int N) {  
    if (row == N) return true;
```

```
    for (int col = 0; col < N; col++) {  
        if (isSafe(row, col, N)) {  
            board[row][col] = 1;
```



```

        if (solve(row + 1, N)) return true;
        board[row][col] = 0;
    }
}
return false;
}

```

```

int main() {
    int N;
    if (scanf("%d", &N) != 1) return 0;

```

```

    for (int i = 0; i < N; i++) {
        for (int j = 0; j < N; j++) {
            board[i][j] = 0;
        }
    }

```

```

    if (solve(0, N)) {
        for (int i = 0; i < N; i++) {
            for (int j = 0; j < N; j++) {
                printf("%d ", board[i][j]);
            }
            printf("\n");
        }
    }
}

```

```

return 0;
}

```

Status : Wrong

Marks : 0/10

4. Problem Statement

In chess, a knight moves in an "L" shape: two squares in one direction and one square perpendicular to it. From position (0, 0) on an infinite chessboard, a knight can move to one of the following eight positions in a single step:

(1, 2), (-1, 2), (1, -2), (-1, -2), (2, 1), (-2, 1), (2, -1), (-2, -1).

Given a target position (x, y) where both x and y are positive integers, find the minimum number of steps needed for the knight to travel from (0, 0) to (x, y) using BFS (Breadth-First Search).

Beginning from location (0,0), what is the minimum number of steps needed for a knight to get to some other arbitrary location (x,y)?

Input Format

The first line contains two space-separated integers x and y, representing the target position of the knight on the chessboard.

Output Format

Print a single integer representing the minimum number of steps needed for the knight to move from (0, 0) to (x, y).

Refer to the sample output for formatting specifications

Sample Test Case

Input: 1 2

Output: 1

Answer

```
#include <stdio.h>
```

```
int main() {  
    int x, y;  
    scanf("%d %d", &x, &y);
```

```
    int dx[8] = {1, 2, 2, 1, -1, -2, -2, -1};  
    int dy[8] = {2, 1, -1, -2, -2, -1, 1, 2};
```

```
    int queue[100][3];  
    int visited[10][10] = {0};
```

```
    int front = 0, rear = 0;
```

```

queue[rear][0] = 0;
queue[rear][1] = 0;
queue[rear][2] = 0;
rear++;

visited[0][0] = 1;

while (front < rear) {
    int cx = queue[front][0];
    int cy = queue[front][1];
    int dist = queue[front][2];
    front++;

    if (cx == x && cy == y) {
        printf("%d", dist);
        return 0;
    }

    for (int i = 0; i < 8; i++) {
        int nx = cx + dx[i];
        int ny = cy + dy[i];

        if (nx >= 0 && ny >= 0 && nx <= 8 && ny <= 8 && !visited[nx][ny]) {
            visited[nx][ny] = 1;
            queue[rear][0] = nx;
            queue[rear][1] = ny;
            queue[rear][2] = dist + 1;
            rear++;
        }
    }
}

return 0;
}

```

Status : Correct

Marks : 10/10

5. Problem Statement

A customer is shopping in a supermarket and has selected N items, each with a fixed price.

The customer has a discount coupon that can be applied only if the total price of some selected items is exactly equal to a target amount S.

Write a program using Backtracking to find all distinct subsets of item prices whose sum is exactly equal to S.

Each item price can be used only once. If the input contains duplicate prices, the output must not contain repeated subsets.

Formula: A subset is valid if: $p_1 + p_2 + \dots + p_k = S$

Print each valid subset with its elements in ascending order. Print subsets in the order they are discovered by the backtracking algorithm (which processes the sorted array from left to right).

Input Format

The first line contains an integer N representing the number of items.

The second line contains N space-separated integers representing item prices. The prices may include duplicate values.

The third line contains an integer S representing the target bill amount.

Output Format

Print each distinct valid subset on a separate line. Within each subset, print elements in ascending order, each followed by a space (including the last element).

Print subsets in the order they are found during backtracking on the sorted input.

If no valid subset exists, print: No subset found

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 5
5 5 10 15 20
25

Output: 5 5 15

5 20
10 15

Answer

```
#include <stdio.h>
#include <stdlib.h>
```

```
int N, S;
int a[25];
int path[25];
int found = 0;
```

```
int cmp(const void *x, const void *y) {
    return (*(int*)x - *(int*)y);
}
```

```
void printPath(int len) {
    for (int i = 0; i < len; i++) {
        printf("%d ", path[i]);
    }
    printf("\n");
}
```

```
void backtrack(int start, int sum, int len) {
    if (sum == S) {
        found = 1;
        printPath(len);
        return;
    }
```

```
    for (int i = start; i < N; i++) {
```

```
        if (i > start && a[i] == a[i - 1]) continue;
```

```
        if (sum + a[i] > S) break;
```

```
        path[len] = a[i];
```

```
        backtrack(i + 1, sum + a[i], len + 1);
```

```
    }
}
```

```
int main() {  
    scanf("%d", &N);  
  
    for (int i = 0; i < N; i++) {  
        scanf("%d", &a[i]);  
    }  
  
    scanf("%d", &S);  
  
    qsort(a, N, sizeof(int), cmp);  
  
    backtrack(0, 0, 0);  
  
    if (!found) {  
        printf("No subset found");  
    }  
  
    return 0;  
}
```

Status : Correct

Marks : 10/10